



NPTEL Online Certification Course
Applied Accelerated Artificial Architecture
Assignment 7: Detailed Solution
Indian Institute of Technology Guwahati

Question 1

Which parallelism strategy involves replicating the entire model on every GPU while splitting the input batch?

- Pipeline Parallelism
- Tensor Parallelism
- Data Parallelism
- Sharded Parallelism

Correct Answer: **Data Parallelism**

Explanation: In Data Parallelism, the complete model parameters are replicated across all participating GPUs, and each GPU processes a distinct partition of the batch. The computed gradients across all GPUs are then aggregated using an all-reduce operation before updating the shared weights.

Question 2

When training a model is splitted into batch wise manner where we split the input weight matrix, which parallelism strategy is required?

- Data Parallelism
- Pipeline Parallelism
- Tensor Parallelism
- Fully Sharded Data Parallel

Correct Answer: **Tensor Parallelism**

Explanation: Tensor Parallelism splits individual weight matrices and tensor operations across multiple GPUs within a single layer. This allows training massive individual layers that exceed the memory capacity of a single GPU.

Question 3

In a PyTorch DistributedDataParallel (DDP) script, what is the consequence of forgetting to call `sampler.set_epoch(epoch)` inside your training loop?

- The training script hangs indefinitely.
- The model runs out of memory.
- The data is shuffled identically every epoch.
- The gradients fail to synchronize.

Correct Answer: **The data is shuffled identically every epoch.**

Explanation: Calling `sampler.set_epoch(epoch)` updates the random seed of the `DistributedSampler` at the start of each epoch. Without it, the sampler reuses the exact same deterministic ordering every epoch, leading to identical batch sequences and reduced generalization.

Question 4

Which two collective communication primitives combine to form the standard Ring All-reduce operation?

- Broadcast and Reduce
- Reduce-scatter and All-gather
- All-gather and Broadcast
- Scatter and Gather

Correct Answer: **Reduce-scatter and All-gather**

Explanation: Ring All-reduce operates in two sequential phases: first, a Reduce-scatter phase aggregates gradient chunks around the ring so each GPU holds a fully reduced chunk. Then, an All-gather phase broadcasts these reduced chunks back to all GPUs so every rank receives the complete updated gradient.

Question 5

For a Ring All-reduce algorithm with N GPUs and a gradient size of M bytes, what does the per-GPU bandwidth cost "asymptotically" approach as N becomes very large?

- M bytes
- $2M$ bytes
- $N \times M$ bytes
- M / N bytes

Correct Answer: **$2M$ bytes**

Explanation: Each GPU transfers $2 \times \frac{N-1}{N} \times M$ bytes of data during the combined Reduce-scatter and All-gather phases. As N becomes very large ($N \rightarrow \infty$), the factor $\frac{N-1}{N}$ approaches 1, making the total transferred data per GPU asymptotically approach $2M$ bytes.

Question 6

How does DistributedDataParallel (DDP) handle execution across GPUs to ensure true isolation?

- One thread per GPU
- One process per GPU
- One process for all GPUs
- Multiple processes per GPU

Correct Answer: **One process per GPU**

Explanation: PyTorch DDP uses a multi-process model where a dedicated Python process is spawned for each individual GPU device. This process isolation completely avoids Python's Global Interpreter Lock (GIL) bottlenecks and avoids GPU resource contention.

Question 7

When saving a checkpoint in DDP, why must you use `.module.state_dict()` instead of just `.state_dict()`?

- To avoid a 'module.' prefix on every key
- To compress the checkpoint size
- To save the optimizer state
- To sync the weights across ranks

Correct Answer: **To avoid a 'module.' prefix on every key**

Explanation: Wrapping a model in `DistributedDataParallel` prepends a "module." prefix to all layer names in its `state_dict()`. Accessing `.module.state_dict()` retrieves the underlying unwrapped model's state dictionary directly, ensuring key compatibility when loading checkpoints into non-DDP models.

Question 8

In a naive all-reduce implementation with N GPUs and gradient size M , what is the per-GPU communication traffic?

- $O(M)$
- $O(N \cdot M)$
- $O(M/N)$
- $O(N+M)$

Correct Answer: **$O(N \cdot M)$**

Explanation: A naive parameter-server or master-worker all-reduce implementation requires transferring the entire M -byte gradient tensor from all N workers to a central node. Consequently, total communication traffic per worker scales linearly with N , resulting in an $O(N \cdot M)$ overhead.

Question 9

During a multi-node rendezvous using c10d, what is the initial action taken by Rank 0?

- Probes for active NVLink connections
- Broadcasts the initial model weights
- Sends a unique NCCL ID to all active nodes
- Opens a TCP store and waits for connections

Correct Answer: **Opens a TCP store and waits for connections**

Explanation: During PyTorch **c10d** initialization, Rank 0 creates a key-value **TCPStore** server listening on a designated host address and port. All secondary worker ranks connect to this central TCP store during rendezvous to discover peer ranks and exchange network configuration data.

Question 10

Which of the following represents the slowest fallback option for interconnect bandwidth?

- PCIe Gen5
- InfiniBand NDR
- RoCEv2 Ethernet
- TCP sockets

Correct Answer: **TCP sockets**

Explanation: Standard TCP sockets rely on CPU software stack processing and standard network protocols, introducing significant latency and lower bandwidth. High-performance interconnects like InfiniBand NDR, RoCEv2, and PCIe Gen5 utilize hardware direct memory access (RDMA/GPUDirect) to bypass CPU software bottlenecks.

Question 11

What is the primary characteristic of "weak scaling" in distributed training?

- Training a bigger problem with constant wall time.
- Finishing a fixed job faster with more GPUs.
- Reducing the communication overhead to zero.
- Minimizing the batch size per GPU.

Correct Answer: **Training a bigger problem with constant wall time.**

Explanation: Weak scaling evaluates execution efficiency when the workload per GPU (local batch size) is kept fixed as more GPUs are added. The goal is to scale up the total model capacity and dataset size while keeping total execution time constant.

Question 12

According to the PyTorch profiler breakdown, what is the recommended fix for addressing "load imbalance" during training?

- Pinning processes to healthy GPUs.
- Using fewer Python callbacks.
- Ensuring same-shape inputs (pad or drop).
- Tuning NCCL bucket sizes.

Correct Answer: **Ensuring same-shape inputs (pad or drop).**

Explanation: Load imbalance occurs when different GPUs process variable sequence lengths or batch sizes, forcing faster ranks to wait idly at synchronization points for slower ranks. Standardizing input tensor shapes across all ranks via padding or dropping residual samples eliminates execution variance.

Question 13

How is scaling efficiency mathematically calculated for an N-GPU training run?

- $\text{throughput}(N) / \text{throughput}(1)$
- $N * \text{throughput}(N) / \text{throughput}(1)$
- $\text{throughput}(N) / (N * \text{throughput}(1))$
- $\text{throughput}(1) / (N * \text{throughput}(N))$

Correct Answer: **$\text{throughput}(N) / (N * \text{throughput}(1))$**

Explanation: Scaling efficiency measures how close a distributed cluster comes to ideal linear speedup by dividing total throughput on N GPUs by N times single-GPU throughput.

A value of 1.0 (100%) indicates perfect linear scaling without communication overhead losses.

Question 14

Which optimization lever is considered the dominant factor for improving communication efficiency?

- Mixed precision (AMP)
- Hierarchical all-reduce
- Gradient compression
- Larger per-GPU batch + gradient accumulation

Correct Answer: **Larger per-GPU batch + gradient accumulation**

Explanation: Increasing local batch sizes and accumulating gradients over multiple micro-batches decreases the frequency of inter-GPU communication calls. Extending computation time relative to networking time drastically reduces communication bottlenecks and boosts overall GPU compute utilization.

Question 15

If you are training a 70B parameter model where the overall memory is very tight, but individual layers still fit perfectly on a GPU, which tool is recommended?

- Fully Sharded Data Parallel (FSDP)
- DistributedDataParallel (DDP)
- Tensor Parallelism (Megatron)
- 3D Parallelism (Megatron-DeepSpeed)

Correct Answer: **Fully Sharded Data Parallel (FSDP)**

Explanation: FSDP shards model parameters, gradients, and optimizer states across data-parallel ranks, gathering parameters layer-by-layer only when needed during forward and backward passes. This eliminates parameter duplication while allowing large models whose individual layers fit into GPU memory to train efficiently.